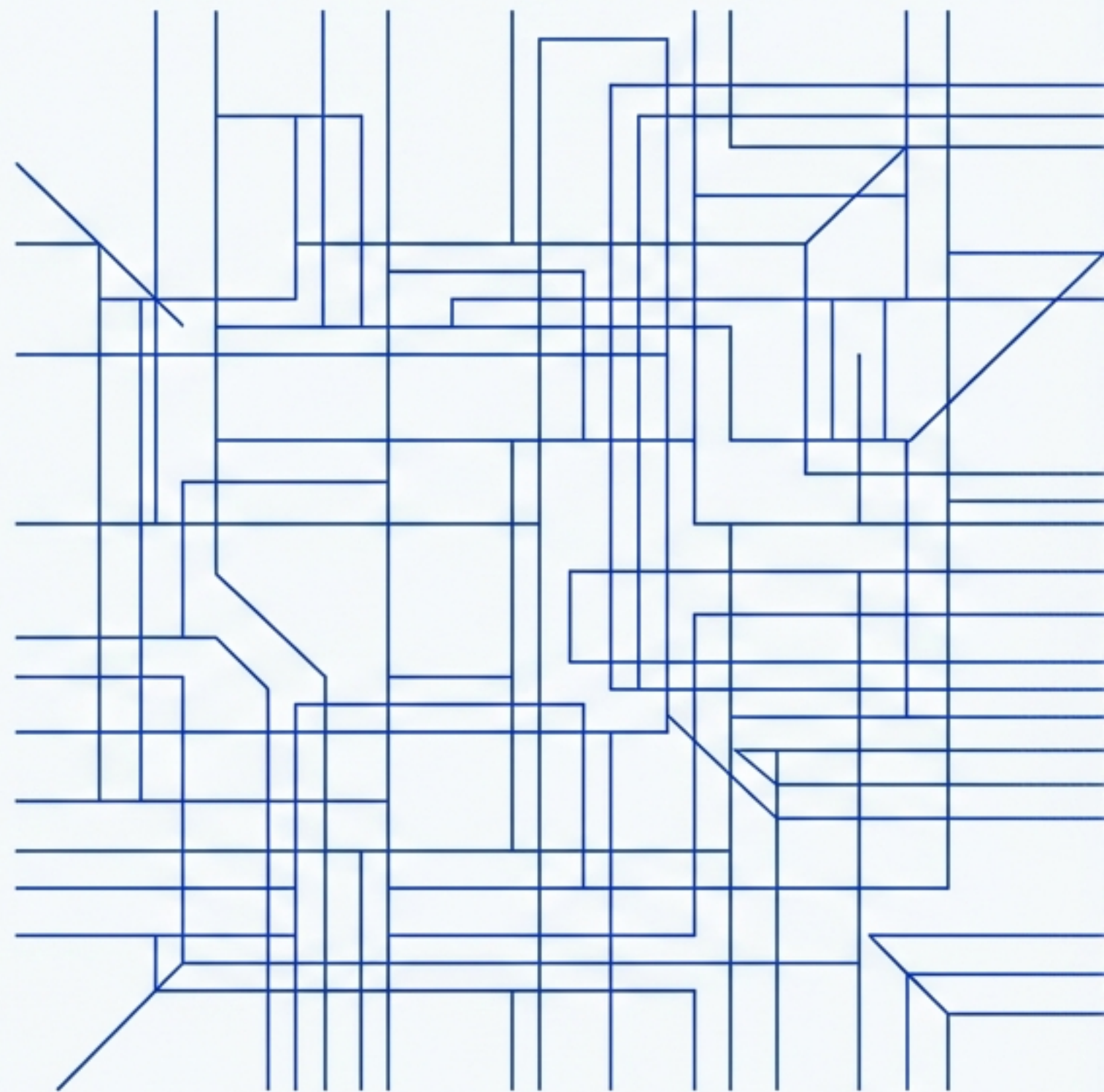


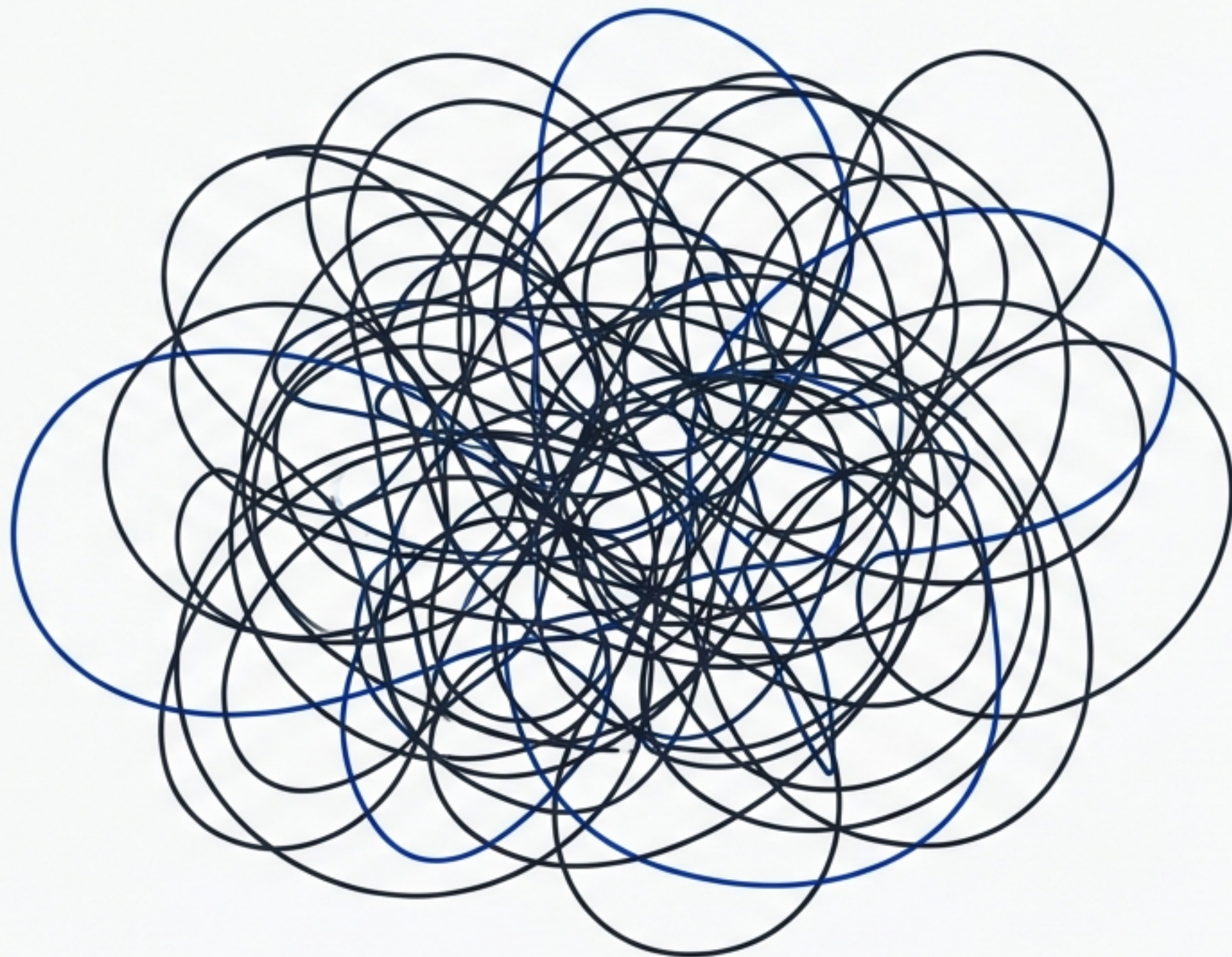
厳格なルールが、最高の開発体験を生む

`strict-rules-nextjs`: 高品質な
Next.jsアプリケーション開発のため
のボイラープレート



プロジェクトの初期設定、カオスになっていませんか？

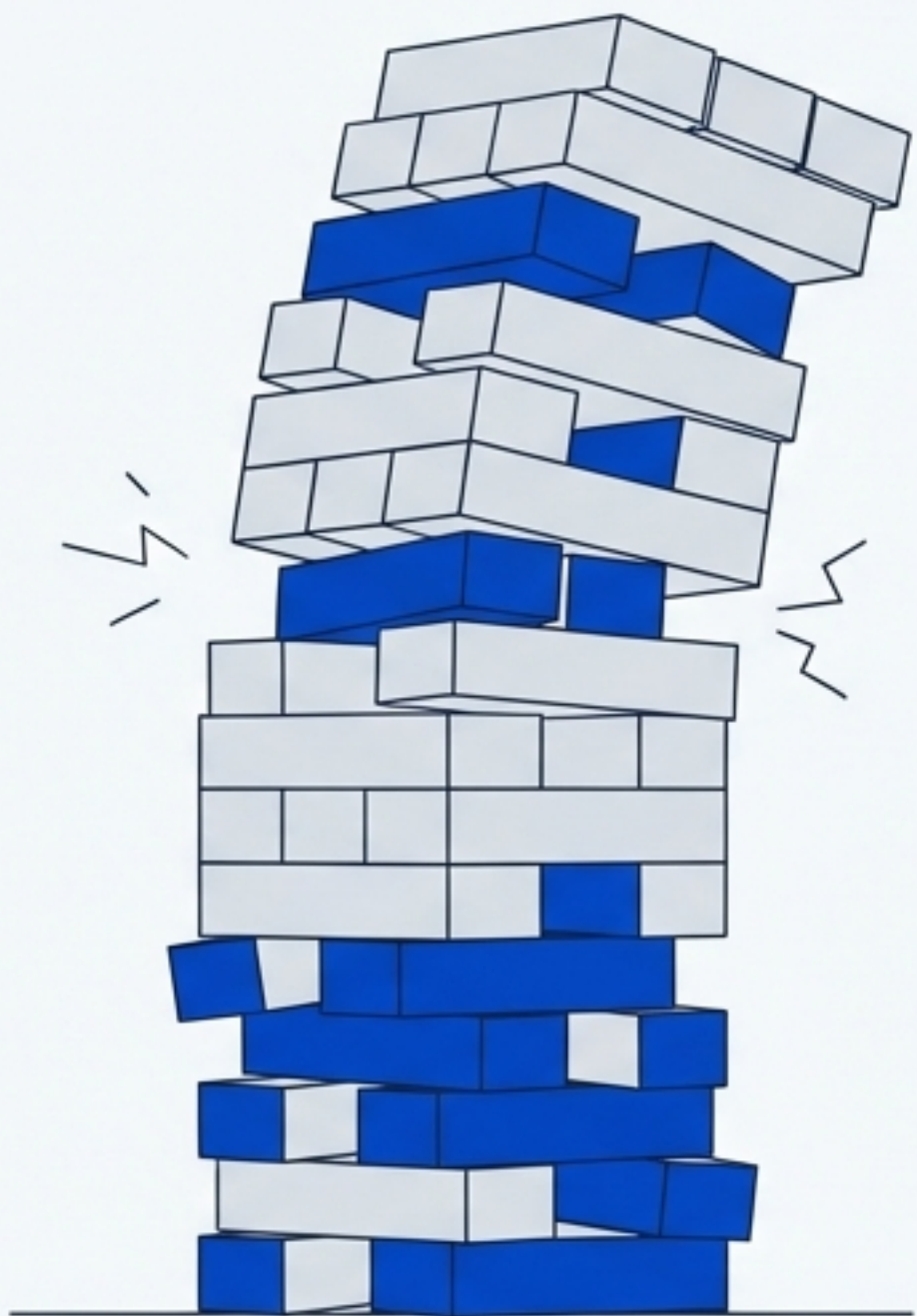
- チームメンバーごとに異なるコーディングスタイル
- レビューで発覚する、些細だが重要なバグ
- コミット履歴に混入するフォーマット修正
- 新しいメンバーが開発を始めるまでの長い準備時間



「小さな妥協」が、技術的負債を積み上げる

- **コードレビューの遅延:** 本質的でないコードレビューの遅延: 本質的でないスタイルや規約の議論に時間が費やされる。
- **デバッグの困難化:** 一貫性のないコードは、バグの根本原因の特定を妨げる。

開発速度の低下: 技術的負債の返済に追われ、新機能の開発が停滞する。



私たちが目指すのは、自動化された「クリーンな開発環境」

- コードは常に一貫性が保たれ、誰が書いても同じスタイルになる。
- 潜在的なバグは、コミットされる前に自動的に検出される。
- 開発者はフォーマットや規約ではなく、ビジネスロジックの実装に集中できる。



その理想を現実にするための答え: `strict-rules-nextjs`

`strict-rules-nextjs`

「`create-next-app`で構築された最新のNext.jsプロジェクトをベースに、最高品質の実践を強制するための厳格なルールセットを事前設定したものです。セットアップの手間を省き、初日から最高のコードを書くことに集中できます。」

品質を守る、4つのガーディアン

プロジェクトの品質を支える主要なツールを、ペルソナ（役割）と共に紹介します。



TypeScript: The Architect (設計者)



ESLint & Prettier: The Inspector (検査官)



Husky: The Gatekeeper (門番)



Vitest: The Validator (検証者)

設計者: TypeScriptによる揺るぎない土台

「このプロジェクトのコードベースは85.8%がTypeScriptで構成されています。これは、単に型を使うということではありません。厳格な型システムを通じて、実行時エラーを未然に防ぎ、自己文書化された保守性の高いコードベースを構築するためのアーキテクチャです。」

```
interface User {  
  id: number;  
  name: string;  
}  
  
function getUserName(user: User): string {  
  return user.name;  
}  
  
const user = { id: 1 }; // Missing 'name' property  
  
// The line below would show a type error  
// Argument of type '{ id: number; }' is not  
// assignable to parameter of type 'User'.  
// Property 'name' is missing.  
getUserName(user);
```


検査官: ESLint & Prettierによる一貫したコード規律

「`eslint.config.mjs`と`prettier.config.mjs`に定義された厳格なルールセットが、すべてのコードを自動的にチェックし、フォーマットします。これにより、コードレビューは本質的なロジックの議論に集中でき、コードベース全体で一貫性が保たれます。」

Before

```
const    hello    = "world";  
function greet (  name){  
    return `Hello, ${ name }!`  
}
```

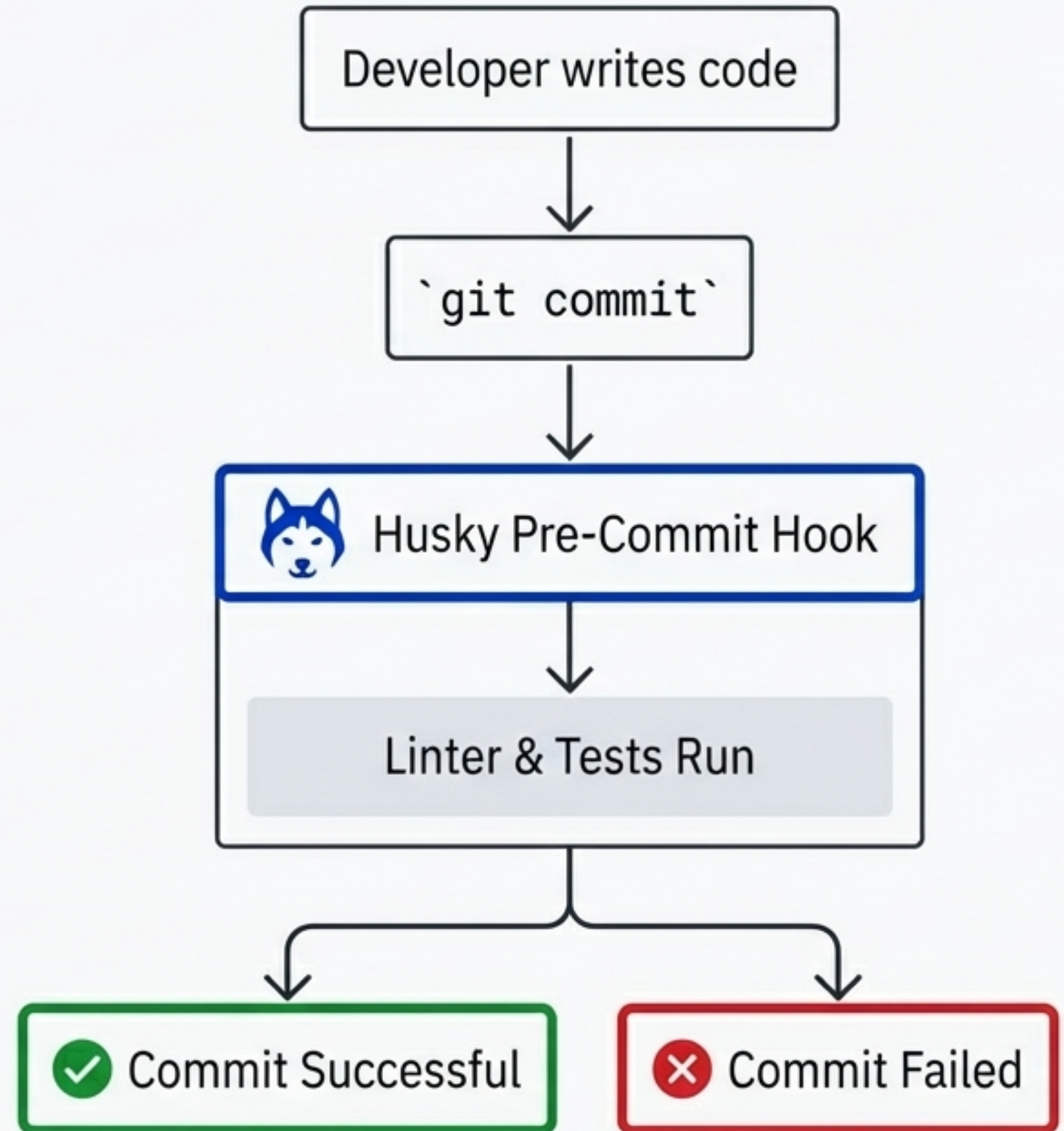


After

```
const hello = 'world';  
  
function greet(name) {  
    return `Hello, ${name}!`;  
}
```


門番： Huskyが低品質なコードの混入を防ぐ

「`git commit`を実行するたびに、Huskyが自動的にLintとテストを実行します。ルールに違反したコードはコミット自体がブロックされるため、リポジトリは常にクリーンな状態に保たれます。これは品質を保証する最後の防衛ラインです。」



検証者: Vitestでロジックの正しさを証明する

「高速なテストフレームワークであるVitestが初期設定済みです。テストを記述する文化を奨励し、アプリケーションのコアロジックが期待通りに動作することを継続的に保証します。設定ファイルは`vitest.config.js`に含まれています。」

```
> Running tests with Vitest...
```

```
PASS src/components/Example.test.ts (2)
```

- ✓ should render the component
- ✓ should handle user interaction correctly

```
PASS src/utils/helpers.test.ts (3)
```

- ✓ should format date string
- ✓ should calculate correctly
- ✓ should return default value on null input

```
Test Suites: 2 passed, 2 total
```

```
Tests: 5 passed, 5 total
```

```
Time: 342ms
```


最新のNext.js App Routerをベースに構築

「基盤は`create-next-app`による最新の構成です。`app/page.tsx`を中心としたApp Routerを採用し、パフォーマンスと開発体験を両立させています。また、`next/font`を通じてVercelの新しいフォントファミリーであるGeistを自動的に最適化・読み込み、細部の品質にもこだわっています。」



なぜ、このボイラープレートを選ぶべきか？



開発速度の向上

面倒な初期設定や規約の議論から解放され、機能開発に集中できます。



チームコラボレーションの円滑化

全員が同じルールに従うため、コードの属人性がなくなり、レビューが効率化します。



品質の自動化

ベストプラクティスがワークフローに組み込まれ、品質が自動的に保証されます。



将来の保守性

一貫性のあるクリーンなコードは、将来の機能追加やリファクタリングを容易にします。

いますぐ始めよう

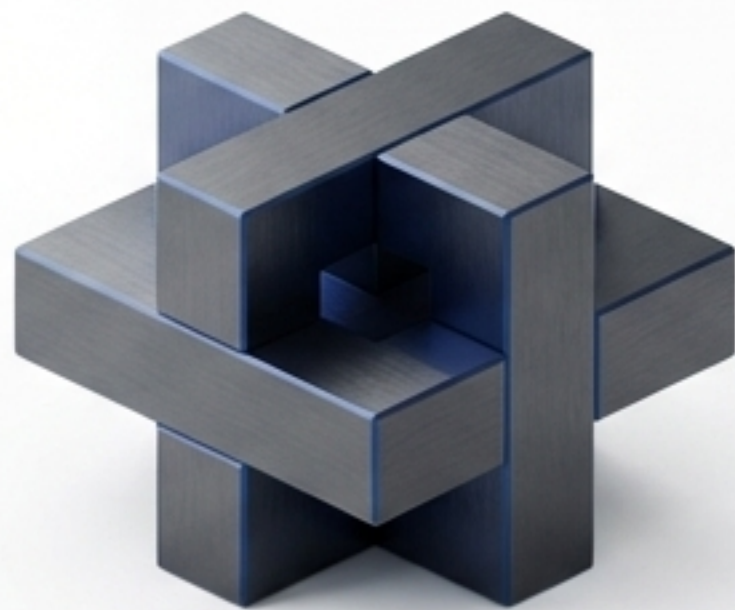
開発サーバーを起動するまでの3つの簡単なステップ。

```
$ git clone https://github.com/masakinihirota/strict-rules-nextjs.git  
$ cd strict-rules-nextjs && npm install  
$ npm run dev
```

`yarn dev`、`pnpm dev`、`bun dev`にも対応しています。ブラウザで <http://localhost:3000> を開いてください。

規律が、創造性を解放する

「厳格なルールは制約ではありません。それは、些細な問題から開発者を解放し、真に価値のある創造的な作業に集中させるための土台です。」



github.com/masakinihirota/strict-rules-nextjs